

Цели реализации:

Создается список `res`, который используется как стек, и индекс элемента из `popres`. Потом мы идём по `pushes` и каждый новый элемент добавляется в `res` и после каждого добавления проверяем последний элемент `res` и если он равен `popres[j]`, то удаляем из `res` и $j++$, делаем так, пока элементы равны. В конце проверяем если достигли конца `popres`, то все ок.

Пример:

pushed: [1 2 3 4 5]

pepper: [1 3 5 4 2]

1) $j=0$ $res = [1] = \text{popped}[0] \Rightarrow res = []$

2) $j = 1$ $res = [2] \neq popped[1]$

3) $j = 1$, $\text{res} = [2, 3]$, $i = \text{popper}[1] \Rightarrow \text{res} = [2]$

4) $j=2$ $res=[2,4]$, $4 \neq popped[2]$
5) $j=2$ $res=[2,4,5]$, $5 = popped[2]$
 $j=3$ $4 = popped[3]$
 $j=4$ $5 = popped[4]$
 $res = []$, $j = len(popped) - 1 \Rightarrow$
 $True$

Оценки сложности

- 1) По времени $O(n)$, где n — кол-во эл-тов в `pushed`, т.к. проходим по всему `pushed`. Т.к. в цикле `while` выполняется не более n раз, то сложность $O(n)$
- 2) По памяти $O(n)$, т.к. в худшем случае добавим в `res` все элементы `pushed`

